

LAB SESSION 09

Implementation of Breadth-First Search (BFS) Algorithm

Objective

- To understand the concept of Breadth-First Search (BFS).
- To implement BFS using a queue data structure.
- To traverse a graph level-by-level.
- To analyze the time and space complexity of BFS.

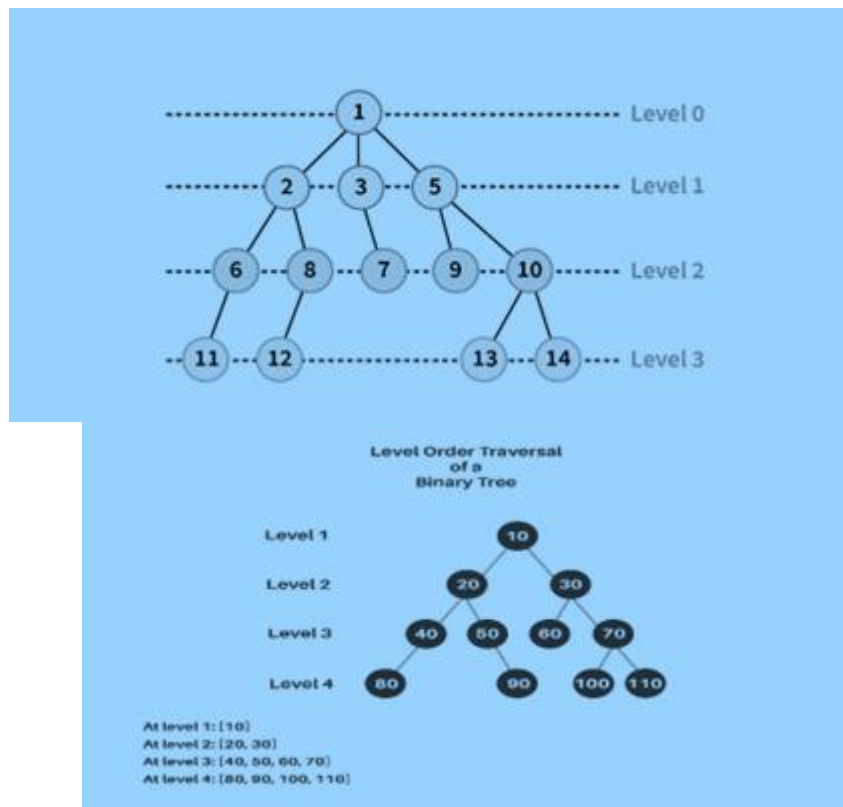
Introduction

Breadth-First Search (BFS) is a graph traversal algorithm that explores nodes **level by level**.

It starts from a source node and visits all its neighbors first, then moves to the neighbors of neighbors.

BFS uses a **Queue (FIFO – First In First Out)** data structure.

Graph Example



If we start BFS from node 0, traversal order will be:

0 → 1 → 2 → 3 → 4 → 5

(Level-by-level traversal)

Key Characteristics

- Traverses graph level-wise
- Uses Queue
- Works for directed and undirected graphs
- Finds shortest path in unweighted graph

Algorithm

1. Start from source node.
2. Mark it as visited.
3. Insert it into the queue.
4. While queue is not empty:
 - Remove front node from queue.
 - Visit all unvisited adjacent nodes.
 - Mark them visited.
 - Add them into the queue.

C++ Program

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

void BFS(vector<vector<int>>& graph, int start) {
    vector<bool> visited(graph.size(), false);
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while(!q.empty()) {
        int node = q.front();
        q.pop();
        cout << node << " ";

        for(int neighbor : graph[node]) {
            if(!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}
```

```

        }
    }
}

int main() {
    vector<vector<int>> graph = {
        {1,2},
        {0,3,4},
        {0,5},
        {1},
        {1},
        {2}
    };

    cout << "BFS Traversal: ";
    BFS(graph, 0);
    return 0;
}

```

Complexity

- **Time Complexity:** $O(V + E)$
- **Space Complexity:** $O(V)$

Where:

V = Vertices

E = Edges

Applications

- Shortest path in unweighted graph
- Network broadcasting
- Social network analysis
- Finding connected components
- Web crawling
- GPS navigation systems

Task 1:

Modify the BFS program to:

- Accept graph input from the user.
- Allow the user to enter the starting vertex.
- Print the BFS traversal.

Task 2:

Modify the BFS algorithm to:

- Find the shortest distance of all vertices from the source node.
- Print the distance of each vertex.

(Hint: Use an additional array `distance[]` and initialize it with -1.)